Practical No. 1

Creation of Test Suite Using Selenium IDE

Aim

To install Selenium IDE, create a test suite containing four test cases for validating different web page formats (HTML, XML, JSON, and Dynamic Form), execute the suite, and export the automation script in WebDriver format.

Learning Objectives

After completing this practical, students will be able to:

1. Install Selenium IDE browser extension.
2. Create and manage test cases in Selenium IDE.
3. Validate different web page formats.
4. Execute test suites and analyze results.
5. Export automated test scripts in WebDriver format.

Software Requirements

Software	Version
Google Chrome / Mozilla Firefox	Latest
Selenium IDE Extension	Latest
Internet Connection	Required

Test URLs Used

Page Type	URL
HTML Page	https://example.com
XML Response	https://httpbingo.org/xml

Name: Palak Singh Roll no. 202403123 TYCS-B STQA

Page Type	URL
JSON API	https://reqres.in/api/users?page=2
Dynamic Form	https://www.selenium.dev/selenium/web/web-form.html

---

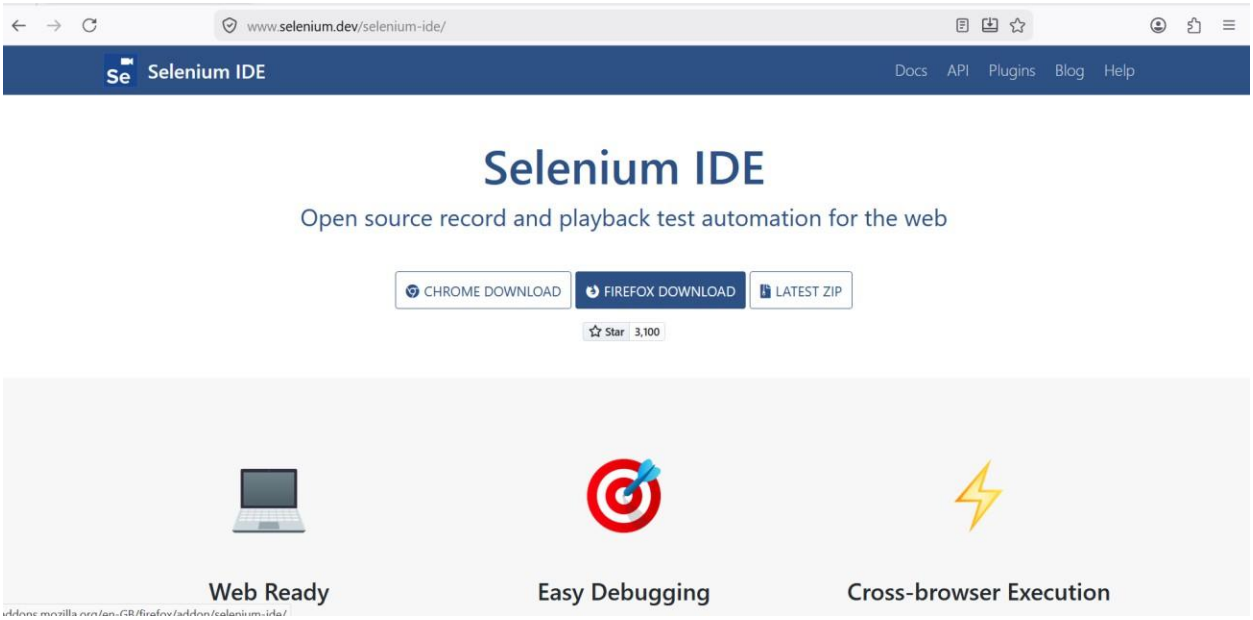
Part A: Installing Selenium IDE

Step 1: Open Firefox Web Store

1. Launch Firefox.
2. Search for **Selenium IDE Firefox Extension**.

Screenshot 1

Insert Screenshot: Selenium IDE page in Firefox Web Store



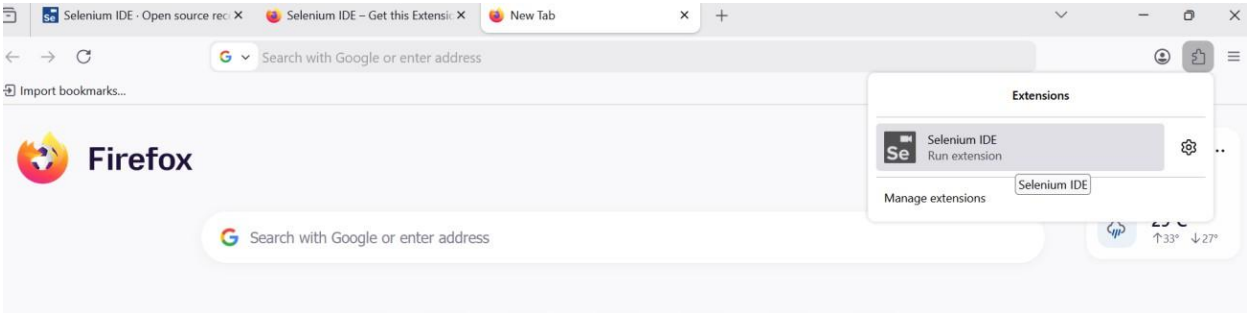
Step 2: Install Extension

1. Click **Add to Firefox**.
2. Click **Add Extension**.
3. Wait for installation to complete.

Name: Palak Singh Roll no. 202403123 TYCS-B STQA

Screenshot 2

Insert Screenshot: Selenium IDE successfully installed



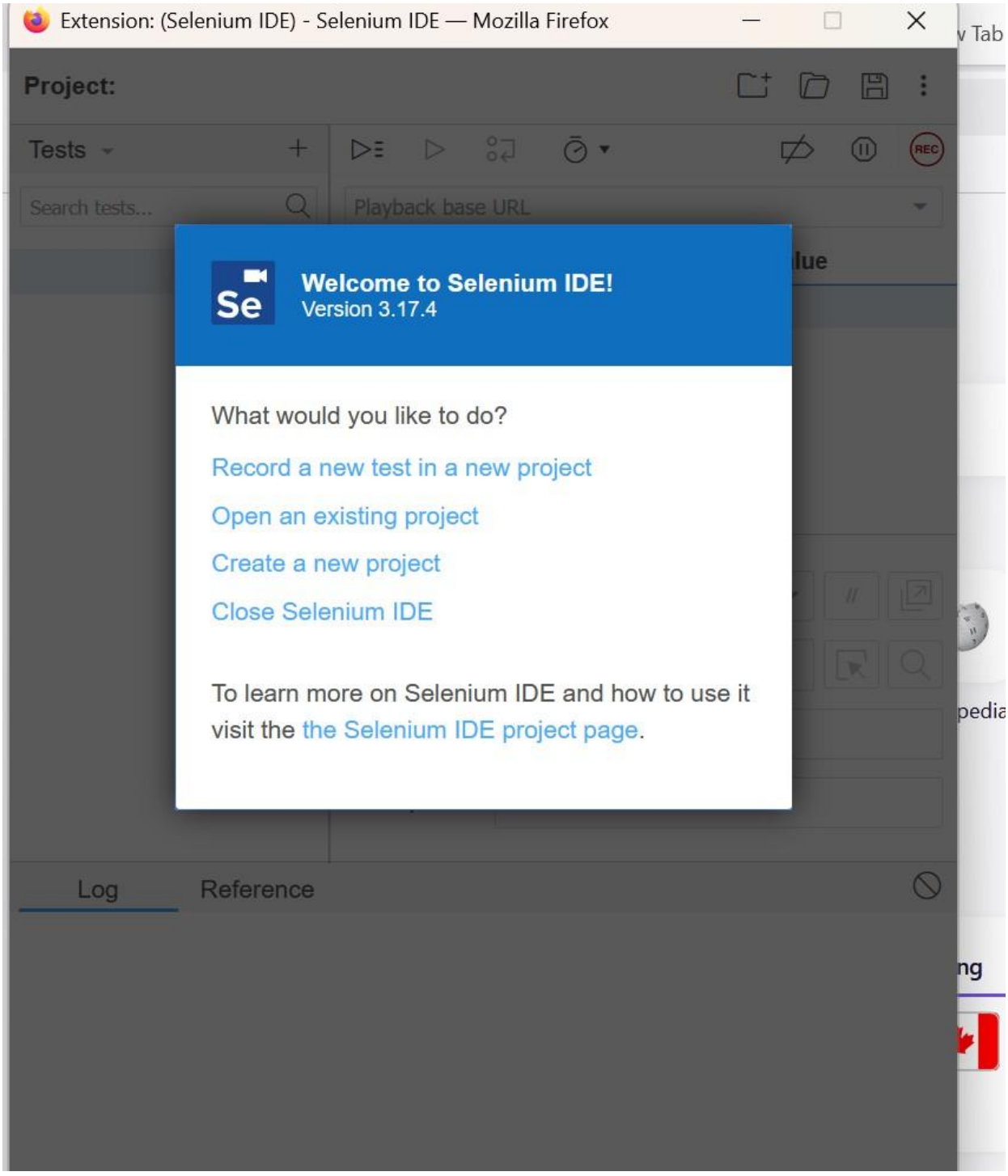
Step 3: Launch Selenium IDE

1. Click Extensions icon.
2. Select Selenium IDE.

Screenshot 3

Insert Screenshot: Selenium IDE startup screen

Name: Palak Singh Roll no. 202403123 TYCS-B STQA



Part B: Create New Project

Step 4: Create Project

1. Click **Create new Project**.
2. Enter Project Name: WebFormatValidation
3. Set Base URL:https://example.com
4. Enter Command = Open, Target = https://example.com.

Since these are **absolute URLs**, Selenium IDE ignores the project's Base URL for that command.

Part C: Create Test Case 1 – HTML Page Validation

Objective

Validate that the HTML page loads successfully.

URL

https://example.com

Start recording. Perform a few actions on the website and stop recording. Save the test case

Step 5: Give Test Case name

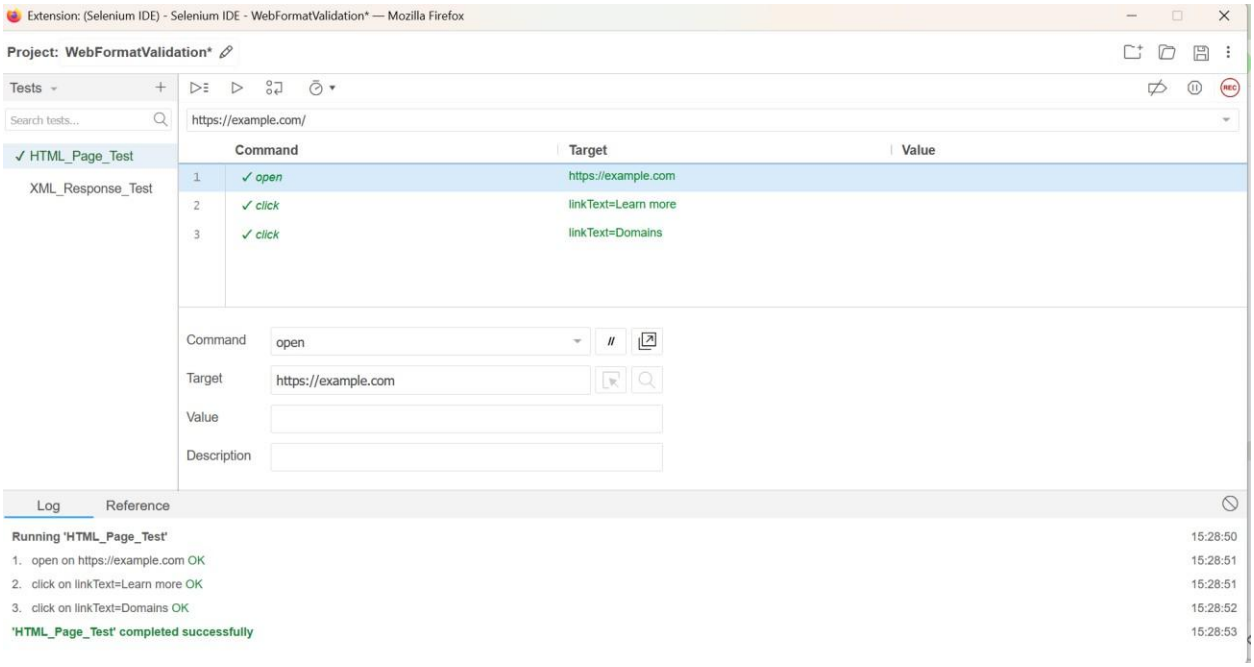
1. Name: HTML_Page_Test

Run the test.

Screenshot 4

Insert Screenshot: HTML Test Case Commands

Name: Palak Singh Roll no. 202403123 TYCS-B STQA



Part D: Create Test Case 2 – XML Response Validation

Objective

Validate XML response page.

URL

https://httpbingo.org/xml

Step 7: Create XML Test

Click + → Add Test to add a new test.

Test Name:
XML_Response_Test

Name: Palak Singh Roll no. 202403123 TYCS-B STQA

Enter URL - https://httpbingo.org/xml

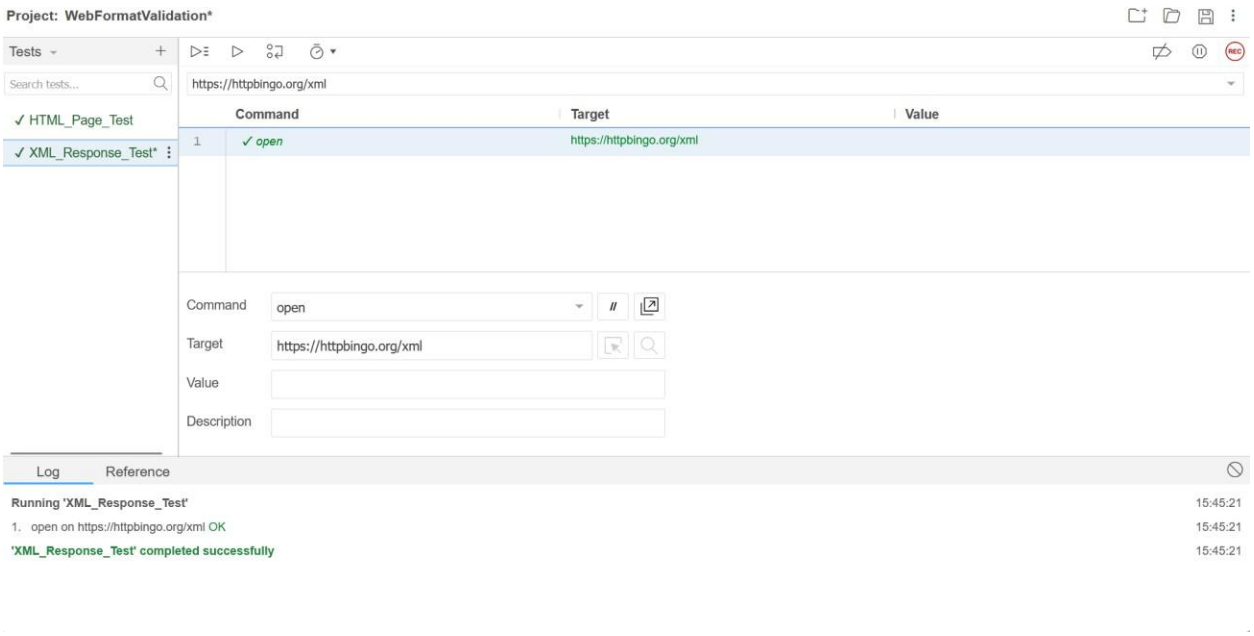
Enter Command = Open, Target = https://httpbingo.org/xml

Start recording. Go to Selenium IDE and stop recording. Save.

Run the test.

Screenshot 6

Insert Screenshot: XML Test Case



Part E: Create Test Case 3 – JSON API Validation

Objective

Validate JSON output.

URL

https://reqres.in/api/users?page=2

Step 8: Create JSON Test

Add a new test. Test Name: JSON_API_Test

Enter Command = Open, Target = https://reqres.in/api/users?page=2

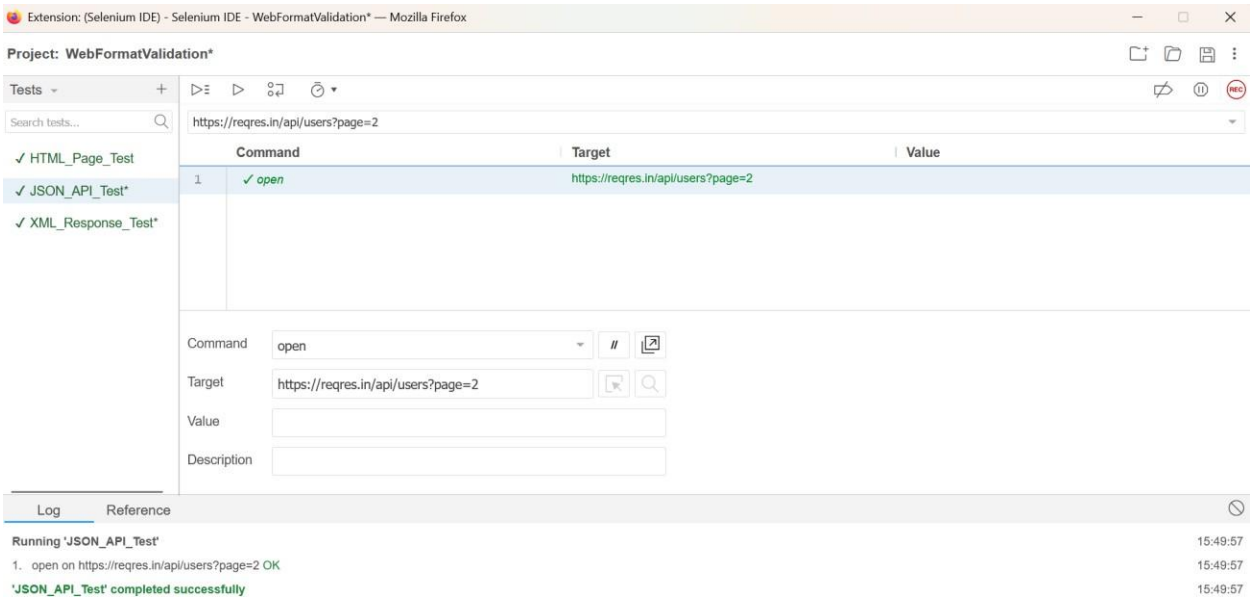
Start recording. Go to Selenium IDE and stop recording. Save.

Run the test.

Students should verify that JSON data appears in the browser.

Screenshot 7

Insert Screenshot: JSON API Test Case



Part F: Create Test Case 4 – Dynamic Form Validation

Name: Palak Singh Roll no. 202403123 TYCS-B STQA

Objective

Validate a dynamic web form.

URL

https://www.selenium.dev/selenium/web/web-form.html

Step 9: Create Dynamic Form Test

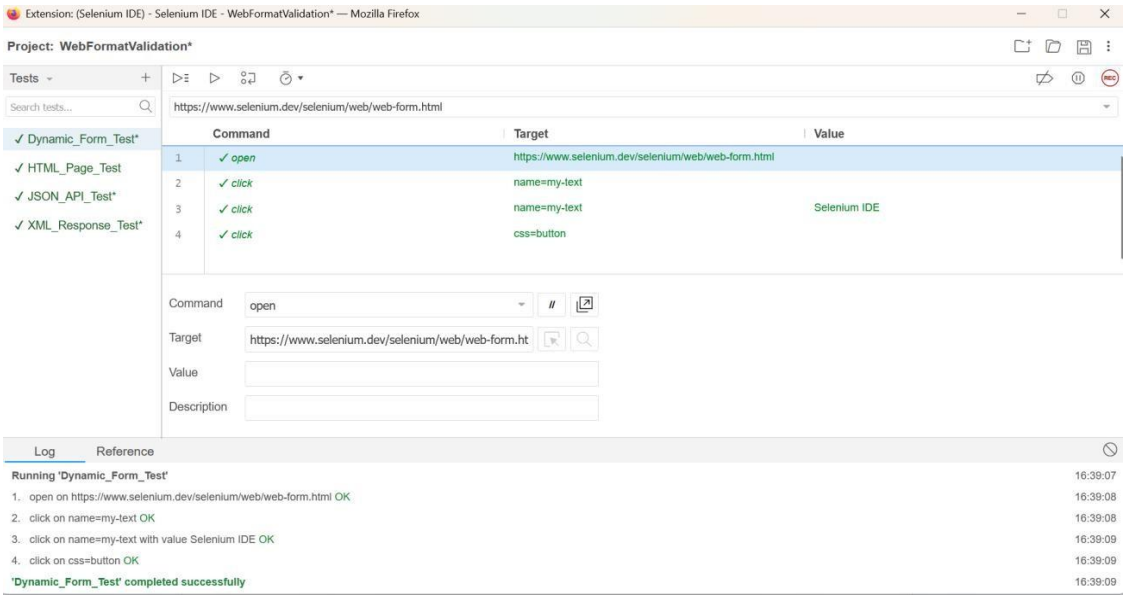
Add a new test. Test Name: Dynamic_Form_Test
Enter below Commands

	Command	Target	Value
1	✓ *open*	https://www.selenium.dev/selenium/web/web-form.html	
2	✓ *click*	name=my-text	
3	✓ *click*	name=my-text	Selenium IDE
4	✓ *click*	css=button	

Run the test.

Screenshot 8

Insert Screenshot: Dynamic Form Test Commands



Name: Palak Singh Roll no. 202403123 TYCS-B STQA

Part G: Create Test Suite

Step 10: Create Suite

1. Click **Test Suites**.
2. Select **New Test Suite**.
3. Name it:

WebPageValidationSuite

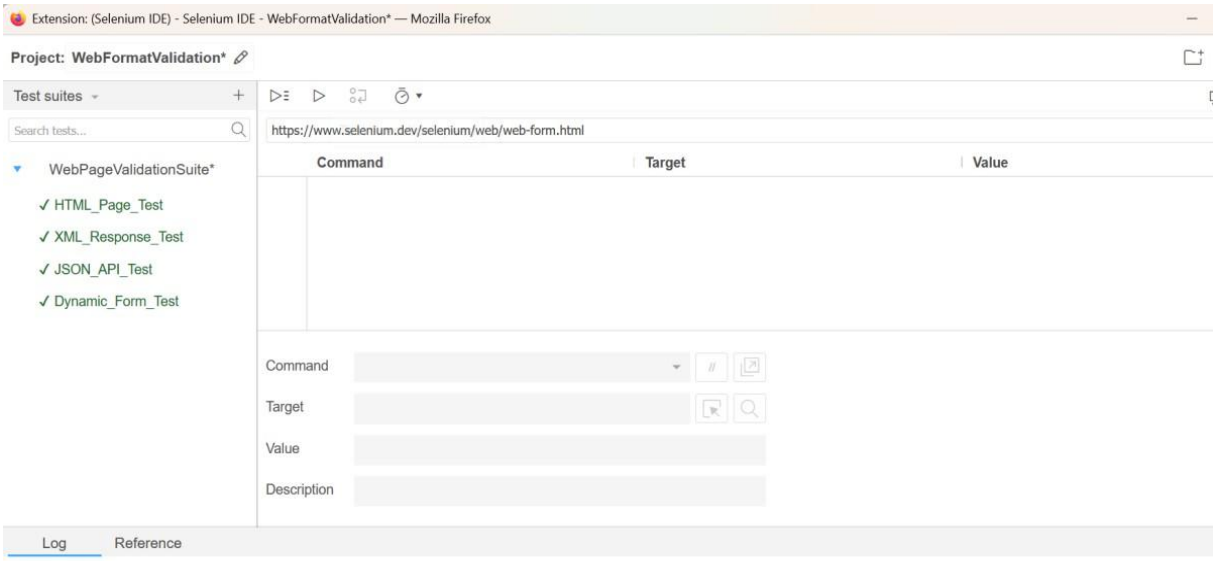
Step 11: Add Test Cases

Add:

- HTML_Page_Test
- XML_Response_Test
- JSON_API_Test
- Dynamic_Form_Test

Screenshot 9

Insert Screenshot: Test Suite with all four test cases



Name: Palak Singh Roll no. 202403123 TYCS-B STQA

Part H: Execute Test Suite

Step 12: Run Suite

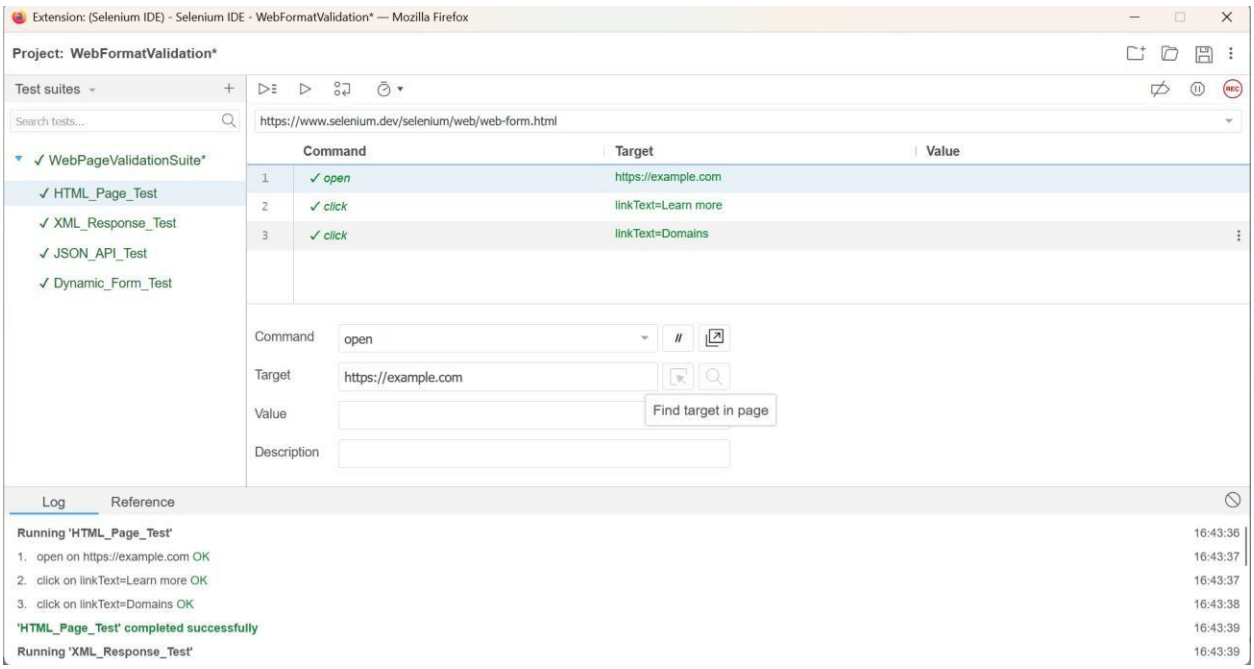
1. Select the Suite.
2. Click **Run Current Test Suite**.

Expected Result

All test cases should pass successfully.

Screenshot 10

Insert Screenshot: Test Execution Results (Pass Status)



Name: Palak Singh Roll no. 202403123 TYCS-B STQA

Part I: Export Script in WebDriver Format

Step 13: Export Project

1. Click Project Suite
2. Select: Export
3. Choose:

Java JUnit WebDriver

or

Python WebDriver

or

JavaScript WebDriver

4. Save the exported script.

Sample Exported Java Junit Code

```
// Generated by Selenium IDE import
org.junit.Test; import org.junit.Before;
import org.junit.After;
import static org.junit.Assert.*;
import static org.hamcrest.CoreMatchers.is; import static
org.hamcrest.core.IsNot.not; import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.firefox.FirefoxDriver; import
org.openqa.selenium.chrome.ChromeDriver;  import
org.openqa.selenium.remote.RemoteWebDriver; import
org.openqa.selenium.remote.DesiredCapabilities; import
org.openqa.selenium.Dimension;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.interactions.Actions;
importorg.openqa.selenium.support.ui.ExpectedConditions; import
org.openqa.selenium.support.ui.WebDriverWait; import
org.openqa.selenium.JavascriptExecutor;
importorg.openqa.selenium.Alert; import
org.openqa.selenium.Keys; import java.util.*;
importjava.net.MalformedURLException; import java.net.URL;
public class WebPageValidationSuiteTest { private WebDriver
    driver;
    private Map<String, Object> vars; JavascriptExecutor js;
    @Before
    public void setUp() {
        driver = new FirefoxDriver();
```

```
    js = (JavascriptExecutor) driver; vars = new
    HashMap<String, Object>();
}
@After
public void tearDown() { driver.quit();
}
@Test
public void hTMLPageTest() { driver.get("https://example.com");
    driver.findElement(By.linkText("Learn more")).click();
    driver.findElement(By.linkText("Domains")).click();
}
@Test
public void xMLResponseTest() { driver.get("https://httpbingo.org/xml");
}
@Test
public void jSONAPITest() { driver.get("https://reqres.in/api/users?page=2");
}
@Test
public void dynamicFormTest() { driver.get("https://www.selenium.dev/selenium/web/web-
    form.html"); driver.findElement(By.name("my-text")).click(); driver.findElement(By.name("my-
    text")).click(); driver.findElement(By.cssSelector("button")).click();
```

Name: Palak Singh **Roll no. 202403123** **TYCS-B** **STQA**

Practical No. 2

Title

Cross-Website Functional Testing Using Selenium IDE

Aim

To develop and execute a Selenium IDE test suite for two different websites by performing common functional operations such as clicking links, filling forms, selecting dropdown options, and verifying displayed content using Selenium IDE in Mozilla Firefox.

Learning Objectives

After completing this practical, students will be able to:

- Install and configure Selenium IDE in Mozilla Firefox.
- Create a Selenium IDE project and test suite.
- Record and edit test cases for multiple websites.
- Perform functional testing using Selenium IDE.
- Execute the test suite successfully.
- Analyze execution logs and test results.

Software Requirements

- **Mozilla Firefox (Latest Version)**
- **Selenium IDE Firefox Add-on**
- Internet Connection

Theory

Functional testing verifies whether a web application behaves according to its specified requirements. Selenium IDE is a browser extension that allows users to record, edit, and replay automated test cases without writing code. It is suitable for beginners who are learning software testing concepts and automation.This practical demonstrates functional testing across two different websites using Selenium IDE.

Name: Palak Singh **Roll no. 202403123** **TYCS-B** **STQA**

Websites Used

Website 1

https://www.wikipedia.org

Website 2

https://the-internet.herokuapp.com

Part A – Install Selenium IDE in Mozilla Firefox

Step 1

Open **Mozilla Firefox**.

Step 2

Visit the **Firefox Add-ons (Mozilla Add-ons) Store**.

Step 3

Search for **Selenium IDE**.

Step 4

Click **Add to Firefox**.

Step 5

When prompted, click **Add** to install the extension.

Step 6

After installation, click the **Extensions** icon in Firefox and select **Selenium IDE**.

Step 7

The Selenium IDE window will open in a new tab.

Name: Palak Singh Roll no. 202403123 TYCS-B STQA

Part B – Create a New Project

Step 1

Open **Selenium IDE**.

Step 2

Click **Create a New Project**.

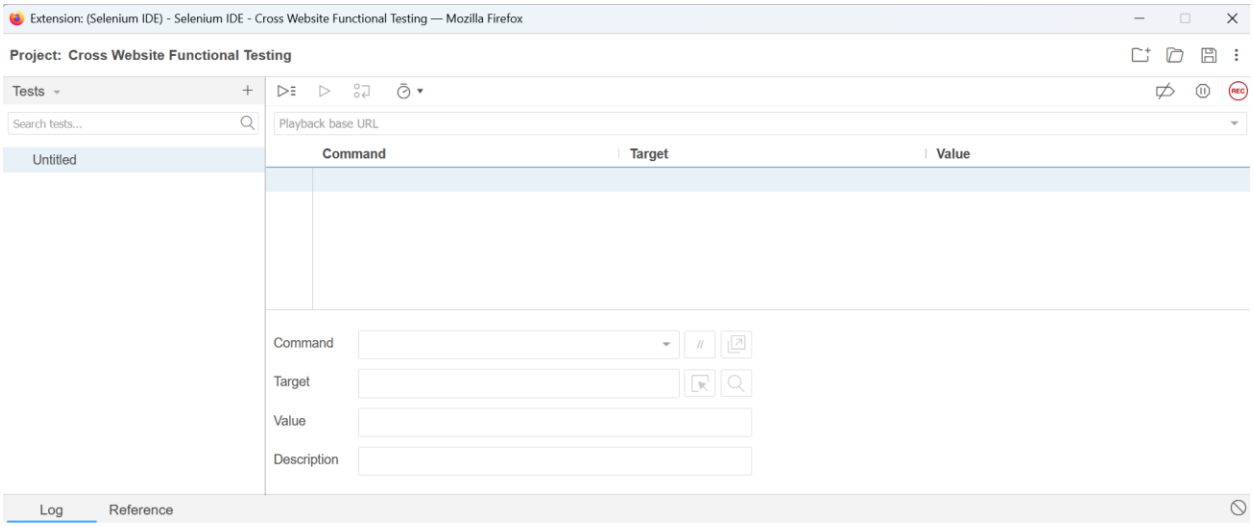
Step 3

Enter the project name:

Cross Website Functional Testing

Step 4

Click **OK**.



Part C – Test Case 1 (Wikipedia)

Step 1

Click **Add New Test**.

Name: Palak Singh Roll no. 202403123 TYCS-B STQA

Test Name

Wikipedia_Test

Step 2

Set the Base URL:

https://www.wikipedia.org

Step 3

Click **Record a New Test**.

Step 4

Perform the following actions:

Action 1

Open the Wikipedia website.

Expected Result: Wikipedia home page opens successfully.

Action 2

Click inside the Search box.

Expected Result: Cursor appears in the search field.

Action 3

Enter the text:

Software Testing

Action 4

Click the **Search** button.

Expect, ed Result: The “Software Testing” article is displayed.

Action 5

Verify that the page heading contains:

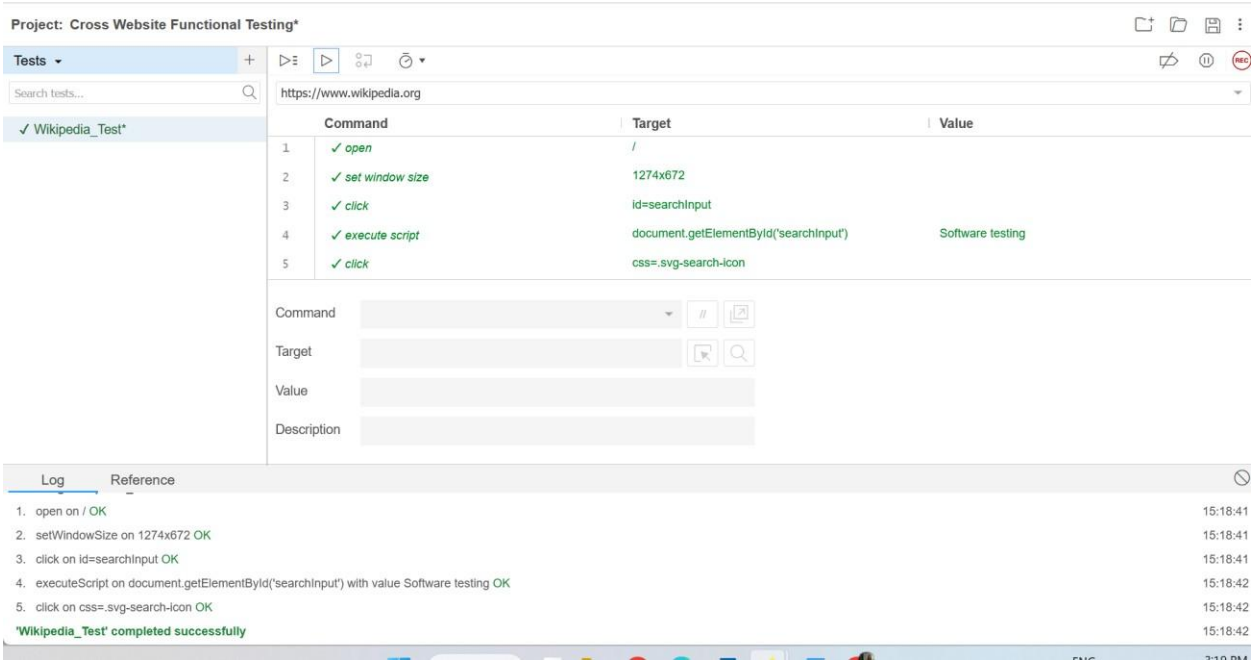
Software testing

Action 6

Click **Stop Recording**.

Instead of the command = type, target = searchInput and value = Software testing, enter -

command = execute script, target = document.getElementById('searchInput') and value= Software testing



Part D – Test Case 2 (The Internet Heroku App)

Step 1

Click **Add New Test**.

Name: Palak Singh **Roll no. 202403123** **TYCS-B** **STQA**

Test Name

Dropdown_Test

Step 2

Set the Base URL:

https://the-internet.herokuapp.com

Step 3

Click **Record a New Test**.

Step 4

Click the **Dropdown** link.

Expected Result: Dropdown page opens.

Step 5

Select **Option 1** from the dropdown list.

Expected Result: Option 1 is selected.

Step 6

Select **Option 2** from the dropdown list.

Expected Result: Option 2 is selected.

Step 7

Return to the Home page.

Step 8

Click **Checkboxes**.

Expected Result: Checkboxes page opens.

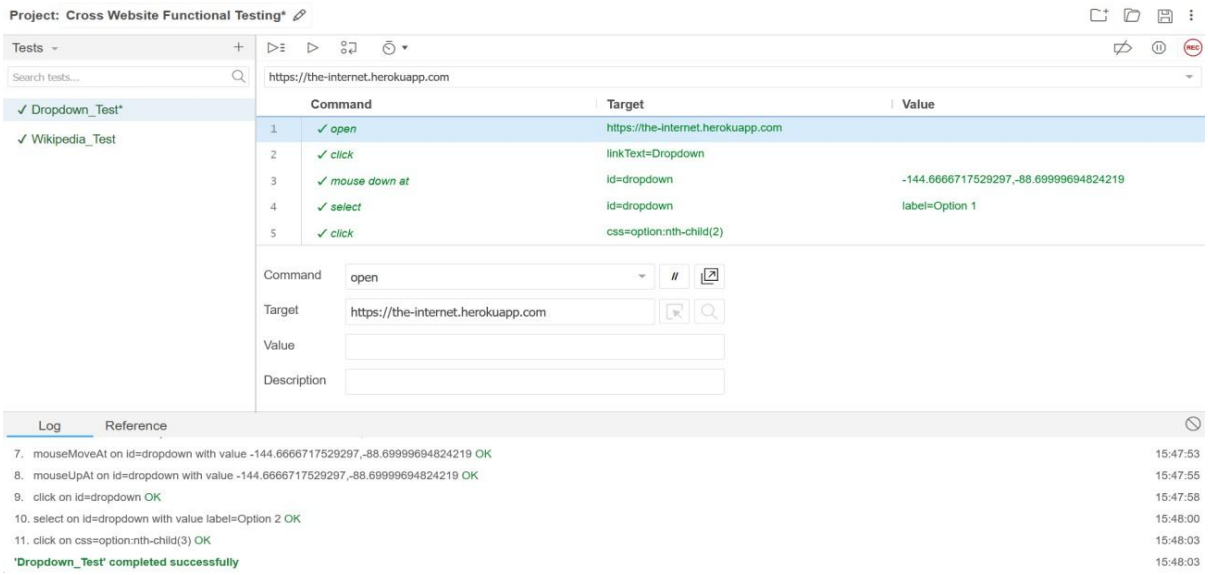
Step 6

Select the first checkbox.

Expected Result: Checkbox is checked successfully.

Step 10

Click **Stop Recording**. Run the test.



Part E – Create a Test Suite

Step 1

Click **Create a New Test Suite**.

Name: Palak Singh Roll no. 202403123 TYCS-B STQA

Suite Name

Functional_Test_Suite

Step 2

Add the following test cases to the suite:

- Wikipedia_Test
- Dropdown_Test

Part F – Execute the Test Suite

Step 1

Click **Run All Tests**.

Step 2

Observe the execution status:

- 🟢 Green indicates **Test Passed**
- 🔴 Red indicates **Test Failed**

Step 3

Wait until all test cases finish execution.

Part G – Analyze Execution Logs

After execution, examine the following details:

- Commands executed
- Target elements
- Input values
- Pass/Fail status
- Execution time

- Error messages (if any)

Expected Output

- The Wikipedia search executes successfully.
- Dropdown options are selected correctly.
- Checkbox selection is successful.
- All test cases execute without errors.
- Execution logs display successful test execution.

Sample Test Result Table

Test Case	Description	Expected Result	Actual Result	Status
TC01	Search "Software Testing" on Wikipedia	Article opens	Article displayed	PASS
TC02	Select Option 1	Option selected	Option selected	PASS
TC03	Select Option 2	Option selected	Option selected	PASS
TC04	Select Checkbox	Checkbox checked	Checkbox checked	PASS

Result

The Selenium IDE project was successfully created and executed in **Mozilla Firefox**. Functional testing was performed on two different websites by executing actions such as searching, clicking links, selecting dropdown values, checking checkboxes, and verifying page content. The execution logs were analyzed to confirm the successful completion of all test cases.

Practical No. 3

Manual Test Case Design and Execution

Aim

To prepare a Test Plan, Test Scenarios, Test Cases, execute the test cases manually, and prepare a Test Execution Report and Defect Report following the Software Testing Life Cycle (STLC).

Practical Outcome

After completing this practical, students will be able to:

- Understand the STLC process.
- Create a Test Plan.
- Design Test Scenarios and Test Cases.
- Execute test cases manually.
- Record test execution results.
- Report software defects in a structured format.

Software Required

- Microsoft Excel / Google Sheets
- Any Web Browser (Chrome/Firefox/Edge)
- Internet Connection

Web Application Used

Use the demo website:

https://practicetestautomation.com/practice-test-login/

This website is designed for software testing practice.

Name: Palak Singh Roll no. 202403123 TYCS-B STQA

STLC Phases Covered

Requirement Analysis
↓
Test Planning
↓
Test Case Development
↓
Test Environment Setup
↓
Test Execution
↓
Defect Reporting
↓
Test Closure

Step 1 – Open the Test Website

Open your browser.

Navigate to

https://practicetestautomation.com/practice-test-login/

You will see the Login page.

Step 2 – Study the Application

Observe the page carefully.

Identify

- Username field
- Password field
- Submit button
- Error message
- Success page

These are the features that will be tested.

Step 3 – Prepare the Test Plan

Open Microsoft Excel.

Name: Palak Singh Roll no. 202403123 TYCS-B STQA

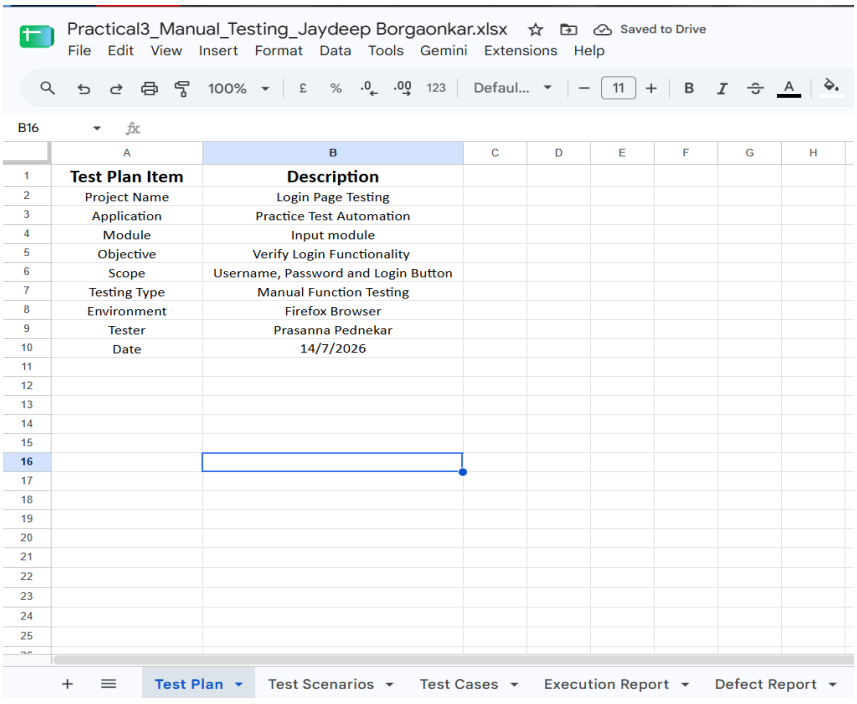
Create Sheet 1.

Name it

Test Plan

Create the following table.

Test Plan Item	Description
Project Name	Login Page Testing
Application	Practice Test Automation
Module	Login
Objective	Verify Login Functionality
Scope	Username, Password and Login Button
Testing Type	Manual Functional Testing Environment
	Windows 10/11, Chrome Browser Tester
	Student Name
Date	Current Date



Step 4 – Prepare Test Scenarios

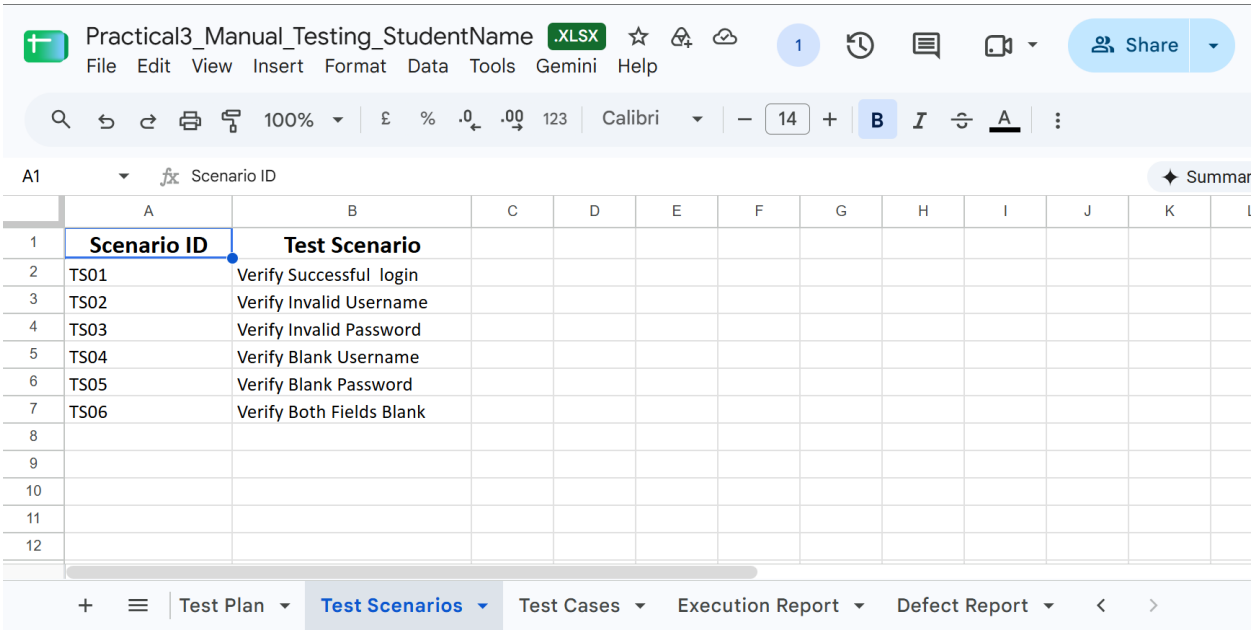
Create Sheet 2.

Name it

Test Scenarios

Create the following table.

Scenario ID	Test Scenario
TS01	Verify successful login
TS02	Verify invalid username
TS03	Verify invalid password
TS04	Verify blank username
TS05	Verify blank password
TS06	Verify both fields blank



Step 5 – Prepare Test Cases

Create Sheet 3.

Name it

Test Cases

Create the following columns.

TC ID Scenario Steps Test Data Expected Result

Test Case 1

Field	Value
TC ID	TC01
Scenario	Valid Login
Steps	Enter valid username, enter valid password, click Login
Test Data	student / Password123
Expected Result	Login successful

Test Case 2

Field	Value
TC ID	TC02
Scenario	Invalid Username
Steps	Enter wrong username, valid password
Test Data	abc / Password123
Expected Result	Error message displayed

Test Case 3

Field	Value
TC ID	TC03
Scenario	Invalid Password
Steps	Enter valid username, wrong password
Test Data	student / abc123 Expected
Result	Error message displayed

Test Case 4

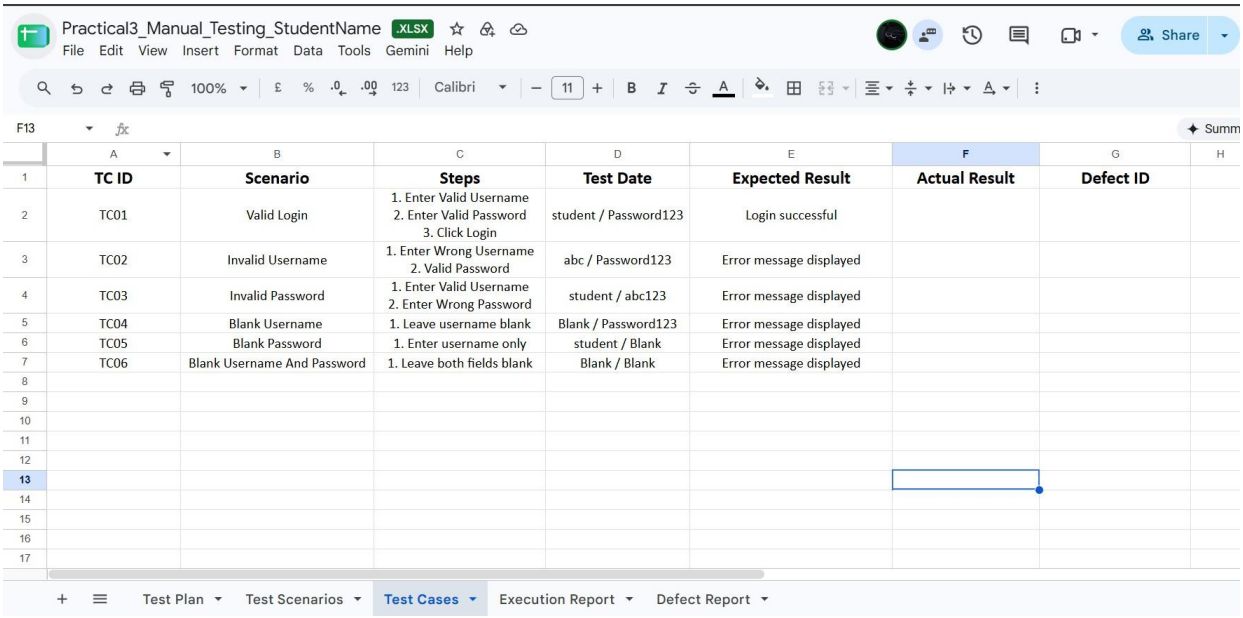
Field	Value
TC ID	TC04
Scenario	Blank Username
Steps	Leave username blank
Test Data	Blank / Password123
Expected Result	Error message displayed

Test Case 5

Field	Value
TC ID	TC05
Scenario	Blank Password
Steps	Enter username only
Test Data	student / Blank Expected
Result	Error message displayed

Test Case 6

Field	Value
TC ID	TC06
Scenario	Blank Username and Password
Steps	Leave both fields blank
Test Data	Blank
Expected Result	Error message displayed



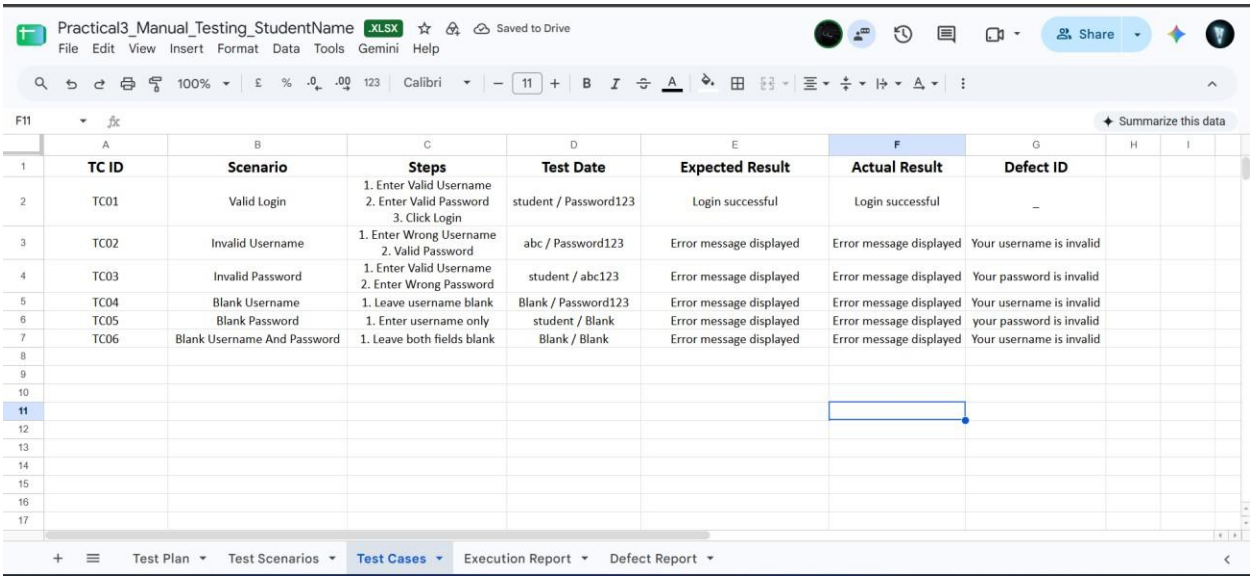
TC ID	Scenario	Steps	Test Date	Expected Result	Actual Result	Defect ID
TC01	Valid Login	1. Enter Valid Username 2. Enter Valid Password 3. Click Login	student / Password123	Login successful		
TC02	Invalid Username	1. Enter Wrong Username 2. Valid Password	abc / Password123	Error message displayed		
TC03	Invalid Password	1. Enter Valid Username 2. Enter Wrong Password	student / abc123	Error message displayed		
TC04	Blank Username	1. Leave username blank	Blank / Password123	Error message displayed		
TC05	Blank Password	1. Enter username only	student / Blank	Error message displayed		
TC06	Blank Username And Password	1. Leave both fields blank	Blank / Blank	Error message displayed		

Step 6 – Execute Test Cases

Run each test manually.

Compare the Actual Result with the Expected Result.

Record the observations.



TC ID	Scenario	Steps	Test Date	Expected Result	Actual Result	Defect ID
TC01	Valid Login	1. Enter Valid Username 2. Enter Valid Password 3. Click Login	student / Password123	Login successful	Login successful	_
TC02	Invalid Username	1. Enter Wrong Username 2. Valid Password	abc / Password123	Error message displayed	Error message displayed	Your username is invalid
TC03	Invalid Password	1. Enter Valid Username 2. Enter Wrong Password	student / abc123	Error message displayed	Error message displayed	Your password is invalid
TC04	Blank Username	1. Leave username blank	Blank / Password123	Error message displayed	Error message displayed	Your username is invalid
TC05	Blank Password	1. Enter username only	student / Blank	Error message displayed	Error message displayed	your password is invalid
TC06	Blank Username And Password	1. Leave both fields blank	Blank / Blank	Error message displayed	Error message displayed	Your username is invalid

---

Step 7 – Prepare Test Execution Report

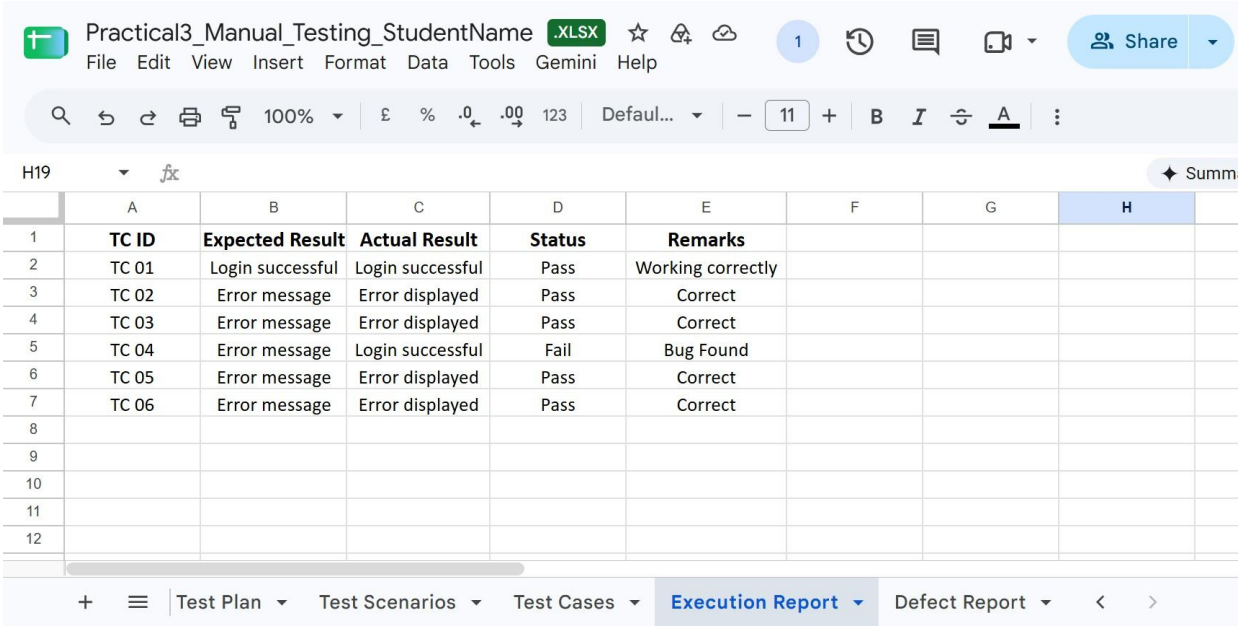
Create Sheet 4.

Name it

Execution Report

Create the following table.

TC ID	Expected Result	Actual Result	Status	Remarks
TC01	Login successful	Login successful	Pass	Working correctly
TC02	Error message	Error displayed	Pass	Correct
TC03	Error message	Error displayed	Pass	Correct
TC04	Error message	Login successful	Fail	Bug Found
TC05	Error message	Error displayed	Pass	Correct
TC06	Error message	Error displayed	Pass	Correct



	TC ID	Expected Result	Actual Result	Status	Remarks
2	TC 01	Login successful	Login successful	Pass	Working correctly
3	TC 02	Error message	Error displayed	Pass	Correct
4	TC 03	Error message	Error displayed	Pass	Correct
5	TC 04	Error message	Login successful	Fail	Bug Found
6	TC 05	Error message	Error displayed	Pass	Correct
7	TC 06	Error message	Error displayed	Pass	Correct

Step 8 – Prepare Defect Report

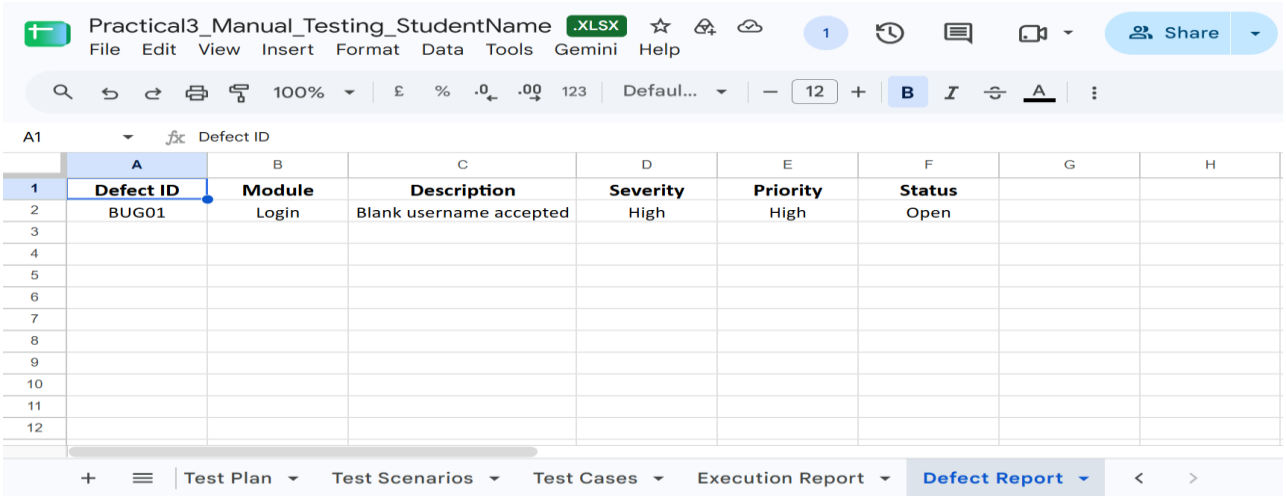
Create Sheet 5.

Name it

Defect Report

Create the following table.

Defect ID	Module	Description	Severity	Priority	Status
BUG01	Login	Blank username accepted	High	High	Open



	Defect ID	Module	Description	Severity	Priority	Status
2	BUG01	Login	Blank username accepted	High	High	Open

Severity Levels

- Critical
- High
- Medium
- Low

Priority Levels

- High
- Medium
- Low

Defect Status

- New
- Open
- Assigned
- Fixed
- Retest
- Closed

Step 9 – Save the Workbook

Save the Excel file as

Practical3_Manual_Testing_StudentName.xlsx

Sample Workbook Structure

Practical3_Manual_Testing.xlsx

```
├── Sheet1 – Test Plan
├── Sheet2 – Test Scenarios
├── Sheet3 – Test Cases
├── Sheet4 – Test Execution Report
└── Sheet5 – Defect Report
```

Practical No. 4: Login Automation and Validation using Selenium WebDriver (Java) with GitHub Codespaces

---

Aim

To automate the login functionality of a web application using Selenium WebDriver (Java) and validate successful login by verifying the URL, success message, and Logout button using assertions.

Learning Outcomes

After completing this practical, students will be able to:

- Access a cloud-based Java development environment.
- Understand the structure of a Selenium Maven project.
- Write Selenium WebDriver code in Java.
- Locate web elements using Selenium locators.
- Validate application behaviour using assertions.

Requirements

Hardware

- Desktop/Laptop
- Internet Connection

Software

- Google Chrome / Microsoft Edge
- GitHub Account

No installation of Java, Eclipse, Selenium libraries, or ChromeDriver is required.

Website Under Test

URL

https://the-internet.herokuapp.com/login

Login Credentials

Username

tomsmith

Password

SuperSecretPassword!

Step 1: Sign in to GitHub

1. Open a web browser.
2. Visit **https://github.com**
3. Sign in using your GitHub account.

Step 2: Open the Practical Repository

Repository link -

https://github.com/conceicaodsouza/STQA-Practicals

Just paste the above link in the browser address bar

Step 3: Launch GitHub Codespaces

1. Click the **Code** button.
2. Select the **Codespaces** tab.
3. Click **Create Codespace on main**.
4. Wait for the online Visual Studio Code environment to load.

After a few moments, the editor opens in your browser with the project already configured.

Step 4: Observe the Project Structure

Expand the project folders in the Explorer panel.

Typical structure:

STQA-Selenium-Practicals

```
│
├── src
│   └── test
│       └── java
│           └── practical
│               LoginTest.java
```

```
│
├── pom.xml
│
└── README.md
```

Explanation:

- src – contains Java source files.
- LoginTest.java – Selenium test program.
- pom.xml – Maven configuration containing Selenium dependencies.

In case the pom.xml file is not in the above structure, move it to the proper location.

Enter command in terminal → 1. find .. -name pom.xml

Then RUN

2. mv src/test/java/practical/pom.xml .

Next RUN

3. ls

Step 5: Open the Java Program

Navigate to:

src

↓

test

↓

java

↓

practical

↓

LoginTest.java

The Java source file will open in the editor.

Step 6: Import Selenium Packages

In the Logintest.java file, Verify that the following import statements are present. Else add them below package practical;

```
import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.WebElement;

import org.openqa.selenium.chrome.ChromeDriver;
```

Step 7: Launch the Browser

Create the WebDriver object in the java file below the public class logintest

```
WebDriver driver = new ChromeDriver();
```

Maximize the browser window.

```
driver.manage().window().maximize();
```

Step 8: Open the Login Page

Navigate to the login page.

```
driver.get("https://the-internet.herokuapp.com/login");
```

Step 9: Enter the Username

Locate the username textbox and enter the username.

```
driver.findElement(By.id("username"))
    .sendKeys("tomsmith");
```

Step 10: Enter the Password

Locate the password textbox.

```
driver.findElement(By.id("password"))
    .sendKeys("SuperSecretPassword!");
```

Step 11: Click the Login Button

Locate and click the Login button.

```
driver.findElement(By.cssSelector("button[type='submit']"))
```

```
    .click();
```

Step 12: Validate the Current URL

Store the current URL.

```
String actualURL = driver.getCurrentUrl();
```

Compare it with the expected URL.

```
if(actualURL.equals("https://the-internet.herokuapp.com/secure"))
{
   System.out.println("URL Validation Passed");
}
else
{
   System.out.println("URL Validation Failed");
}
```

Step 13: Validate the Success Message

Read the success message displayed after login.

```
String message =
driver.findElement(By.id("flash")).getText();
```

Validate the message.

```
if(message.contains("You logged into a secure area!"))
{
   System.out.println("Welcome Message Validation Passed");
}
else
{
   System.out.println("Welcome Message Validation Failed");
}
```

Step 14: Validate the Logout Button

Locate the Logout button.

```
WebElement logout =
driver.findElement(By.cssSelector(".button.secondary.radius"));
```

Verify that it is displayed.

```
if(logout.isDisplayed())
{
   System.out.println("Logout Button Visible");
}
else
{
   System.out.println("Logout Button Not Visible");
}
```

Step 15: Close the Browser

Terminate the browser session.

```
driver.quit();
```

In java file replace chromium by google-chrome in the command -
options.setBinary("/usr/bin/google-chrome");

In pom.xml file remove the first line and save.

Step 16: Run the Program

1. Open the **Terminal** in Codespaces.
2. Execute the below commands –

```
sudo apt-get update
sudo apt-get install -y libxkbcommon0 libasound2t64 libgbm1 libxcomposite1 libxdamage1 libxfixes3 libxrandr2 libatspi2.0-0
ldd /home/codespace/.cache/selenium/chrome/linux64/151.0.7922.47/chrome | grep "not found"
```

If everything is installed correctly, this command should produce **no output**.

Now, test chrome –

/home/codespace/.cache/selenium/chrome/linux64/151.0.7922.47/chrome --headless --no-sandbox --disable-dev-shm-usage --disable-gpu --disable-software-rasterizer --disable-extensions --window-size=1920,1080 --dump-dom https://example.com

You should see HTML output

3. Execute the Maven command:

mvn test

or

mvn clean test

Wait for the program to execute.

Expected Output

The automation script should perform the following actions:

- Launch the browser.
- Open the login page.
- Enter the username.
- Enter the password.
- Click the Login button.
- Redirect to the secure page.
- Display the success message.
- Verify the Logout button.
- Close the browser.

Console Output:

URL Validation Passed

Welcome Message Validation Passed

Logout Button Visible

Observation Table

Test Case	Expected Result	Actual Result	Status
Open Login Page	Login page displayed		

Name: Palak Singh Roll no. 202403123 TYCS-B STQA

Test Case	Expected Result	Actual Result	Status
Enter Username	Username accepted		
Enter Password	Password accepted		
Click Login	Login successful		
URL Validation	/secure page displayed		
Success Message	"You logged into a secure area!" displayed		
Logout Button	Logout button visible		

Result

The login functionality was successfully automated using Selenium WebDriver with Java in GitHub Codespaces. The automation script validated the successful login by checking the URL, success message, and visibility of the Logout button. The expected results were obtained.

JAVA CODE (entire code to be pasted in the java file)

```
package practical;

import org.junit.Test;

import static org.junit.Assert.*;
import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.WebElement;

import org.openqa.selenium.chrome.ChromeDriver;

import org.openqa.selenium.chrome.ChromeOptions;

public class LoginTest {

@Test
    public void loginTest() {


        // Configure Chrome for headless execution

        ChromeOptions options = new ChromeOptions();


        options.addArguments("--headless");
```

```
options.addArguments("--no-sandbox");
options.addArguments("--disable-dev-shm-usage");
options.addArguments("--disable-gpu");
options.addArguments("--disable-software-rasterizer");
options.addArguments("--disable-extensions");
options.addArguments("--window-size=1920,1080");

// Launch browser
WebDriver driver = new ChromeDriver(options);

System.out.println("Browser launched successfully");

// Open login page
driver.get("https://the-internet.herokuapp.com/login");

System.out.println("Login page opened");

// Enter username
driver.findElement(By.id("username")).sen
dKeys("tomsmith");
System.out.println("Username entered");
// Enter password
driver.findElement(By.id("password"))
      .sendKeys("SuperSecretPassword!");

System.out.println("Password entered");

// Click Login button
driver.findElement(By.cssSelector("button[type='submit']"))
      .click();

System.out.println("Login button clicked");

// Assertion 1: URL Validation
String actualURL = driver.getCurrentUrl();
assertEquals(
```

```
            "https://the-internet.herokuapp.com/secure",
            actualURL
        );

        System.out.println("URL Validation Passed");

        // Assertion 2: Welcome Message Validation
        String message =
            driver.findElement(By.id("flash")).getText();

        assertTrue(
            message.contains("You logged into a secure area!")
        );
           System.out.println("Welcome Message Validation Passed");


        // Assertion 3: Logout Button Visibility
        WebElement logout =
            driver.findElement(
                By.cssSelector(".button.secondary.radius")
            );

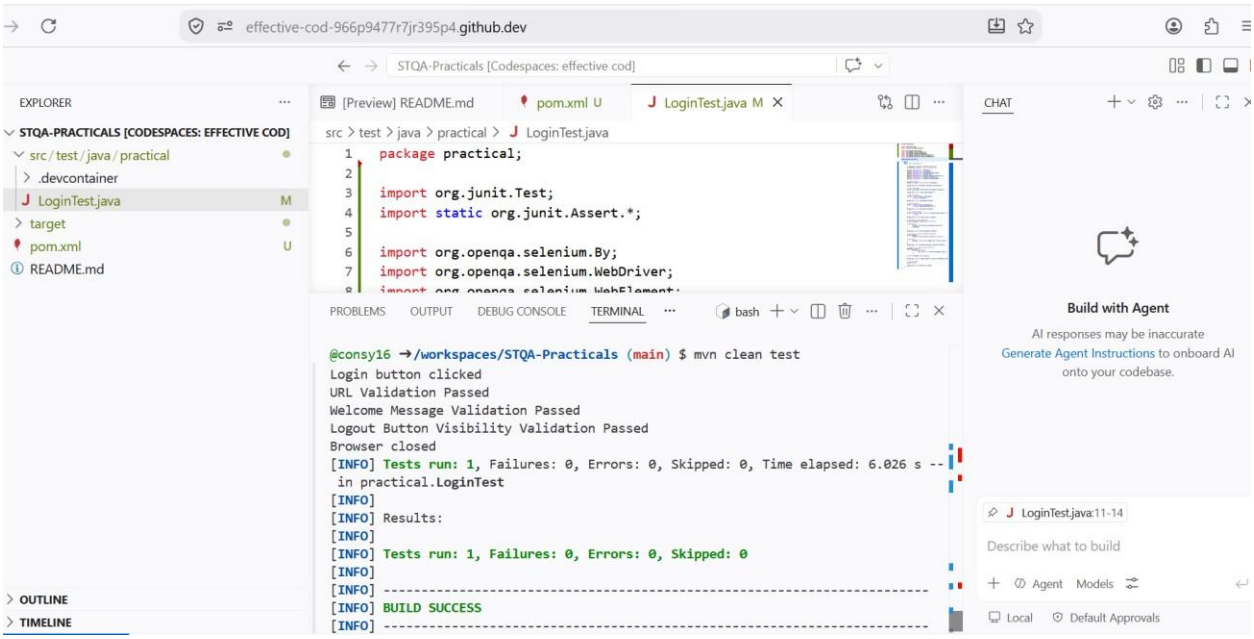

        assertTrue(logout.isDisplayed());


        System.out.println("Logout Button Visibility Validation Passed");


        // Close browser
        driver.quit();


        System.out.println("Browser  closed");
    }
}
```

Name: Palak Singh Roll no. 202403123 TYCS-B STQA



Practical 5

WebElement Interaction Using Selenium WebDriver

Objective

To create an automated test using Selenium WebDriver in Java to interact with different web elements such as:

- Text box
- Dropdown list

The test will also verify the results using assertions.

Requirements

No software installation is required on your computer. The

practical will be performed using:

- GitHub Codespaces
- Java
- Maven
- Selenium WebDriver
- Chromium browser
- ChromeDriver

These tools are already configured in the practical environment.

Step 1: Open the Practical Repository

Sign in to GitHub using your GitHub account.

Open the practical repository link provided by your instructor.

https://github.com/Conceicaodsouza/STQA-WebElement-Practical

Step 2: Open the Repository in GitHub Codespaces

On the repository page:

1. Click the green **Code** button.
2. Select the **Codespaces** tab.
3. Click **Create codespace on main**.

Wait for GitHub Codespaces to finish creating the development environment.

> The first time the Codespace is created, it may take a few minutes.

Step 3: Verify the Project Structure

After the Codespace opens, the project should contain the following structure:

```
STQA-WebElement-Practical
├── .devcontainer
│   └── devcontainer.json
├── pom.xml
└── src
    └── test
        ├── java
        │   └── practical
        │       └── BrowserTest.java
        └── resources
            └── web-elements.html
```

The important files are:

pom.xml src/test/java/practical/BrowserTest.java
src/test/resources/web-elements.html

Step 4: Open the Java Test Program

In the Explorer panel, navigate to:

src
→ test

→ java
→ practical
→ BrowserTest.java

Open the file.

The program uses Selenium WebDriver to:

1. Start the Chromium browser.
2. Open the HTML practical page.
3. Locate a text box.
4. Enter text into the text box.
5. Verify the entered text.
6. Locate a dropdown list.
7. Select an option from the dropdown.
8. Verify the selected option.
9. Close the browser.

Step 5: Understand the Important Selenium Packages

The program imports the following Selenium classes:

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver; import
org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver; import
org.openqa.selenium.chrome.ChromeOptions; import
org.openqa.selenium.support.ui.Select;
```

These classes are used for:

Class	Purpose
WebDriver	Controls the browser
ChromeDriver	Starts the Chromium browser
ChromeOptions	Configures the browser
By	Locates web elements
WebElement	Represents an HTML element
Select	Works with dropdown lists

Step 6: Understand the Browser Configuration

The program uses headless execution:

```
ChromeOptions options = new ChromeOptions();

options.addArguments("--headless=new");
options.addArguments("--no-sandbox"); options.addArguments("--
disable-dev-shm-usage"); options.addArguments("--disable-gpu");
options.addArguments("--window-size=1920,1080");
```

The --headless=newoption allows the browser to run without displaying a graphical browser window.

Therefore, students may not see a browser window opening during execution. This is

expected.

The test still performs all browser operations in the background.

Step 7: Open the HTML Test Page

The program opens the HTML file using:

```
String filePath = System.getProperty("user.dir")
        + "/src/test/resources/web-elements.html";

driver.get("file://" + filePath);
```

The HTML page is already provided in the project. Students do

not need to create or download the HTML file.

Step 8: Verify the Page Title

The program verifies the page title:

```
assertEquals(
        "Web Element Interaction Practical", driver.getTitle());
```

The test passes only if the actual page title is:

Web Element Interaction Practical

This is an example of an assertion

Step 6: Locate the Text Box

The text box is located using its ID:

```
WebElement nameInput = driver.findElement(By.id("name"));
```

The HTML element has:

```
<input id="name">
```

The Selenium statement:

```
By.id("name")
```

locates the element using its idattribute.

Step 10: Enter Text into the Text Box

The following statement enters text:

```
nameInput.sendKeys("Conceicao");
```

The program then verifies that the text was entered correctly:

```
assertEquals(
        "Conceicao", nameInput.getAttribute("value")
);
```

The test checks that:

Expected value: Conceicao Actual value: Conceicao

If both values match, the assertion passes.

Step 11: Locate the Dropdown

The country dropdown is located using:

```
WebElement countryDropdown = driver.findElement(By.id("country"));
```

The HTML element has:

```
<select id="country">
```

The element is then converted into a Selenium Selectobject:

```
Select countrySelect =
```

```
        new Select(countryDropdown);
```

Step 12: Select an Option from the Dropdown

The following statement selects India:

countrySelect.selectByVisibleText("India"); The option is

then verified:

```
assertEquals(
        "India", countrySelect.getFirstSelectedOption().getText()
);
```

This confirms that the correct dropdown option was selected.

Step 13: Close the Browser

The browser is closed using:

```
if (driver != null) { driver.quit();
}
```

This is placed inside a finallyblock so that the browser is closed even if an error occurs during the test.

Step 14: Open the Terminal

In GitHub Codespaces:

1. Click **Terminal** from the top menu.
2. Select **New Terminal** if required.

The terminal should open at the project directory. You

should see a prompt similar to:

@username ➜ /workspaces/STQA-WebElement-Practical (main) $

Step 15: Install Required Tools

Before running the Selenium practical, install Maven, Chromium, and ChromeDriver in the Codespace.

In the **Terminal** at the bottom of the Codespaces window, run: sudo

apt-get update

Wait for the command to finish.

Then run:

sudo apt-get install -y maven chromium chromium-driver Wait

until the installation is complete.

Verify Maven

Run:

mvn -version

You should see output similar to: Apache

Maven 3.8.7

Java version: 21.0.8

Verify Chromium

Run:

chromium --version

You should see output similar to:

Chromium 150.0.7871.181

Verify ChromeDriver

Run:

chromedriver --version

You should see output similar to:

ChromeDriver 150.0.7871.181

Note: The exact version numbers may be different for different students. This is normal. The browser and ChromeDriver versions should be compatible.

Step 16: Run the Test

In the terminal, execute:

mvn test Press

Enter. Maven

will:

1. Compile the Java test program.
2. Download required dependencies if necessary.
3. Start the Chromium browser in headless mode.
4. Execute the Selenium test.
5. Perform the web element interactions.
6. Execute the assertions.
7. Close the browser.
8. Display the test result.

Step 16: Verify Successful Execution

A successful execution should contain:

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

The final result should be:

BUILD SUCCESS

This indicates that the practical was completed successfully.

Expected Result

The test should successfully:

Open the web page
↓
Verify the page title
↓
Locate the name text box
↓
Enter text
↓
Verify entered text
↓
Locate the country dropdown
↓
Select India
↓
Verify selected option
↓

Close the browser

↓

BUILD SUCCESS

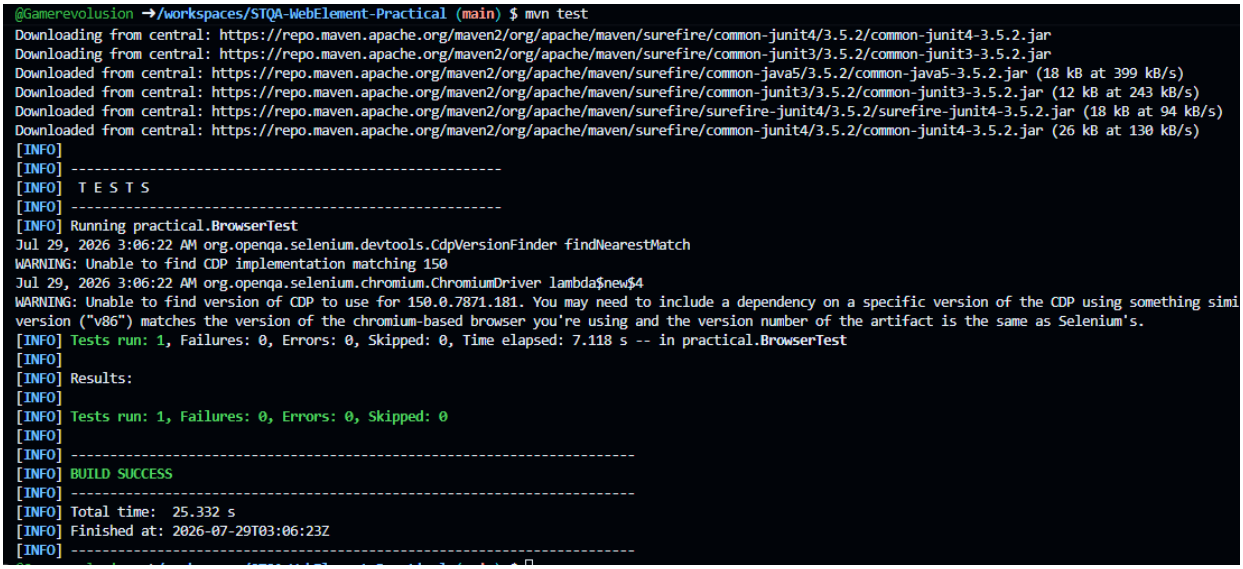
Final Output

The practical is considered successfully completed when the terminal displays:

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

and:

BUILD SUCCESS



Java Code:

Name: Palak Singh Roll no. 202403123 TYCS-B STQA

```
package practical;

import org.junit.Test;
import static org.junit.Assert.*;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver; import
org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver; import
org.openqa.selenium.chrome.ChromeOptions; import
org.openqa.selenium.support.ui.Select;

public class BrowserTest {

    @Test
    public void browserTest() {

        WebDriver driver = null;

        try {
            // Configure Chromium for headless execution ChromeOptions
            options = new ChromeOptions(); options.addArguments("--
            headless=new"); options.addArguments("--no-sandbox");
            options.addArguments("--disable-dev-shm-usage");
            options.addArguments("--disable-gpu"); options.addArguments("--
            window-size=1920,1080");

            // Specify the Chromium browser installed in GitHub Codespaces
            options.setBinary("/usr/bin/chromium");

            // Start the browser
            driver = new ChromeDriver(options);

            // Open the local HTML file
            String filePath = System.getProperty("user.dir")
                    + "/src/test/resources/web-elements.html";

            driver.get("file://" + filePath);

            // Verify page title
```

```
                assertEquals(
                        "Web Element Interaction Practical", driver.getTitle()
                );

                // Locate the text box
                WebElement nameInput = driver.findElement(By.id("name"));

                // Enter text into the text box
                nameInput.sendKeys("Conceicao");

                // Verify the entered text assertEquals(
                        "Conceicao", nameInput.getAttribute("value")
                );

                // Locate the country dropdown WebElement
                countryDropdown =
                        driver.findElement(By.id("country"));

                Select countrySelect =
                        new Select(countryDropdown);

                // Select India from the dropdown countrySelect.selectByVisibleText("India");

                // Verify the selected option assertEquals(
                        "India", countrySelect.getFirstSelectedOption().getText()
                );

            } finally {

                // Close the browser if (driver
                    != null) { driver.quit();
                }
            }
        }
}
```

Practical No. 6 – Data-Driven Testing using Selenium WebDriver and Apache POI

Aim

To perform Data-Driven Testing using Selenium WebDriver by reading student information from an Excel file using Apache POI.

Software Required

- GitHub Account
- GitHub Codespaces
- Internet Connection

Note: No software installation is required on your local computer.

Step 1: Open the Repository

Open the GitHub repository shared by your instructor.

Example:

https://github.com/conceicaodsouza/STQA-Practical-6-DataDrivenTesting

---

Step 2: Create a Codespace

1. Click **Code**.
2. Select the **Codespaces** tab.
3. Click **Create codespace on main**.
4. Wait for the Codespace to load completely.

Step 3: Verify Java Installation

Open the Terminal and execute:

java -version

Expected output:

openjdk version "21"

Step 4: Verify Maven Installation

Execute:

mvn -version

Expected output:

Apache Maven 3.9.x If

you receive:

mvn: command not found install

Maven:

sudo apt-get update

sudo apt-get install -y maven

Verify again:

mvn -version

Step 5: Verify Git LFS

Execute:

git lfs version

If you receive:

git: 'lfs' is not a git command

install Git LFS:

sudo apt-get install -y git-lfs git

lfs install

Verify:

git lfs version

Step 6: Verify Chromium Browser

Execute:

which chromium

If the command returns something similar to:

/usr/bin/chromium Chromium is

installed.

If there is **no output**, install Chromium:

sudo apt-get update

sudo apt-get install -y chromium chromium-driver

Step 7: Verify Chromium Version

Execute:

chromium --version

Example output:

Chromium 151.x.x.x

Step 8: Verify ChromeDriver

Execute:

which chromedriver

Expected output:

/usr/bin/chromedriver

If there is **no output**, install Chromium and ChromeDriver: sudo

apt-get update

sudo apt-get install -y chromium chromium-driver

Step G: Verify ChromeDriver Version

Execute:

chromedriver --version

Example output:

ChromeDriver 151.x.x.x

Step 10: Open the Web Application Folder

In the terminal, execute: cd

webapp

Step 11: Start the Local Web Server

Execute:

python3 -m http.server 8000 Expected

output:

Serving HTTP on 0.0.0.0 port 8000...

Do not close this terminal.

Leave the server running.

Step 12: Open a New Terminal

From the VS Code menu:

Terminal → New Terminal

The first terminal should continue running the Python server.

Step 13: Return to the Project Folder

In the new terminal, execute:

cd ..

Verify that you are back in the project folder:

pwd

Step 14: Run the Selenium Test

Execute:

mvn test

Step 15: Verify Successful Execution

The following messages should appear:

✓ Browser launched successfully.

✓ Student Management System opened successfully.

BUILD SUCCESS

Step 16: Stop the Python Server

Return to the first terminal and press:

Ctrl + C

to stop the local web server.

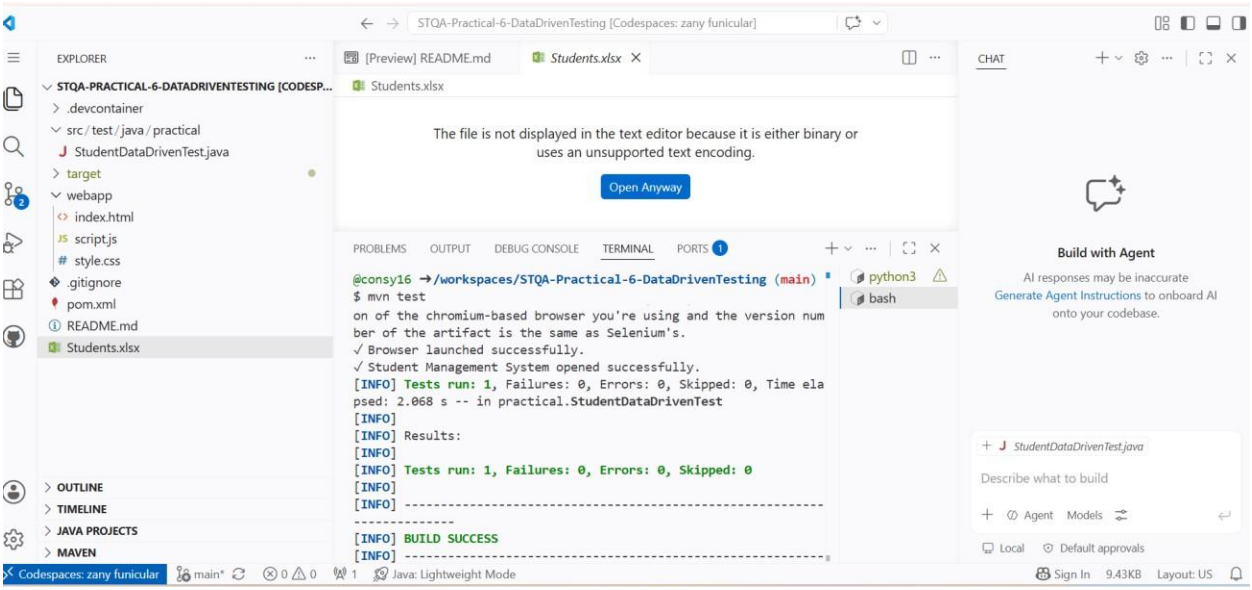
Project Structure

```
STQA-Practical-6-DataDrivenTesting
│
├── Students.xlsx
├── pom.xml
├── src
│   └── test
│       └── java
│           └── practical
│               StudentDataDrivenTest.java
├── webapp
│   ├── index.html
│   ├── style.css
│   └── script.js
└── .devcontainer
```

Common Errors and Solutions

Name: Palak Singh Roll no. 202403123 TYCS-B STQA

Error	Cause	Solution
mvn: command not found	Maven is not installed	sudo apt-get install -y maven
git: 'lfs' is not a git command	Git LFS is not installed	sudo apt-get install -y git-lfs followed by git lfs install
which chromium returns nothing	Chromium is not installed	sudo apt-get install -y chromium chromium-driver
which chromedriver returns nothing	ChromeDriver is not installed	sudo apt-get install -y chromium chromium-driver
ERR_CONNECTION_REFUSED	Python web server is not running	Start the server using python3 -m http.server 8000 in the webapp folder before running mvn test.
BUILD SUCCESS with CDP warning	Selenium does not yet include DevTools support for the installed Chromium version	This warning can be ignored if the test completes successfully.



Name: Palak Singh Roll no. 202403123 TYCS-B STQA

PRACTICAL NO. 7

Data Extraction and Score Analysis from Web Table

TOOL: Selenium WebDriver with Java

ENVIRONMENT: GitHub Codespaces

BUILD TOOL: Maven

DATA: 10 student records

TEACHER REPOSITORY:

https://github.com/Conceicaodsouza/STQA-Practical-7-DataExtraction

1. Aim

To automate the extraction of student marks from a dynamically populated web table using Selenium WebDriver and Java, identify students who have scored above 60 in at least one subject, and calculate the total count and percentage of qualifying students.

2. Problem Statement

Automate a web page containing a dynamic student marks table. Extract the marks of 10 students using Selenium WebDriver, determine whether each student has scored more than 60 in at least one subject, and calculate the total number and percentage.

3. Tools and Technologies

- GitHub account
- GitHub Codespaces
- Java JDK
- Apache Maven
- Selenium WebDriver 4.34.0
- Chromium/Chrome browser and compatible driver
- JUnit 4

4. Teacher Repository

Students must use the repository supplied by the teacher:

```
https://github.com/Conceicaodsouza/STQA-Practical-7-DataExtraction
```

IMPORTANT: Do not create a new repository or change the teacher repository unless instructed.

5. Open the Repository in GitHub Codespaces

1. Open the teacher repository.
2. Sign in to GitHub if required.
3. Click Code.
4. Open the Codespaces tab.
5. Click Create codespace on main.
6. Wait for the Codespace to finish loading.
7. Open Terminal → New Terminal.

6. Verify the Project

Run:

```
pwd
ls
```

The project should contain files such as:

```
pom.xml
students.html
src/
```

IMPORTANT: The most important Maven file is pom.xml.

7. Check Java and Maven

```
java -version
mvn -version
```

IMPORTANT: If these commands work, do not reinstall Java or Maven.

8. If Java or Maven Is Missing

```
sudo apt-get update

sudo apt-get install -y openjdk-21-jdk maven

java -version
mvn -version
```

9. Check Chromium / Chrome

```
which chromium
whichchromium-browser
which google-chrome
which chromedriver
```

Some commands may return no result. A suitable Chromium/Chrome installation must be available.

10. If Chromium Is Missing

```
sudo apt-get update
```

IMPORTANT: Do not download an unrelated ChromeDriver manually.

11. Actual Project Structure

```
STQA-Practical-7-DataExtraction/
├── pom.xml
├── students.html
└── src/
    └── test/
        └── java/
```

The actual teacher repository uses students.html and the Selenium test practical.StudentScoreTest.

12. Web Page Used

The Selenium test opens:

```
http://localhost:8000/students.html
```

The table contains 10 students and three subjects: English, Mathematics and Science. The page is dynamically populated, so Selenium waits for the table rows.

13. Student Data

Roll No	Name	English	Mathematics	Science	Status
101	Aarav	72	55	68	Above 60
102	Diya	45	52	58	Below 60
103	Rohan	81	67	74	Above 60
104	Ananya	55	61	49	Above 60

105	Kabir	39	48	52	Below 60
106	Meera	65	58	62	Above 60
107	Rahul	51	44	59	Below 60
108	Isha	76	69	71	Above 60
109	Arjun	42	57	46	Below 60
110	Sara	63	54	60	Above 60

The condition is strictly greater than 60 (> 60). A mark of exactly 60 alone does not qualify.

14. Working Selenium Program

The teacher repository already contains the working program. Students normally only need to execute it. Its main logic is:

```
driver.get("http://localhost:8000/students.html");


WebDriverWait wait =
    new WebDriverWait(driver, Duration.ofSeconds(10));


wait.until( ExpectedConditions.numberOfElementsToBeMoreThan( By.cssSelector("#studentTable
        tbody tr"), 0
    )
);


List<WebElement> rows =
    driver.findElements(By.cssSelector("#studentTable tbody tr"));


int totalStudents = rows.size(); int
qualifyingStudents = 0;


for (WebElement row : rows) {
    List<WebElement> cells =
        row.findElements(By.tagName("td"));
```

```
        int english = Integer.parseInt(cells.get(2).getText());
        int mathematics = Integer.parseInt(cells.get(3).getText()); int science =
        Integer.parseInt(cells.get(4).getTex
        mathematics > 60 ||
        science > 60;


    if (above60) {
        qualifyingStudents++;

    }
}
```

15. Why a Local Web Server Is Required

The test accesses students.html through localhost port 8000. Therefore a local HTTP server must be running before Selenium starts.

16. CRITICAL: Start the Web Server First

CRITICAL: DO NOT RUN mvn test BEFORE STARTING THE WEB SERVER.

In Terminal 1 run:

```
python3 -m http.server 8000
```

You should see:

```
Serving HTTP on 0.0.0.0 port 8000 ...
```

IMPORTANT: Keep this terminal open. Do not press Ctrl+C.

17. Open a Second Terminal

8. Leave Terminal 1 running.
9. Select Terminal → New Terminal.
10. Use Terminal 2 for Maven.

```
pwd
```

18. Run the Test

```
mvn test
```

Wait for Maven to finish. On the first run, dependencies may be downloaded.

19. Expected Analysis Output

```
========== ANALYSIS RESULT

==========
Total Students: 10

Students scoring above 60 in at least one subject: 6 Percentage:
60.0%

==========
```

20. Expected Maven Result

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 BUILD
SUCCESS
```

21. Calculation

```
Total Students = 10
Qualifying Students = 6



Percentage = (6 / 10) × 100
```

22. Verified Result

The Selenium WebDriver program successfully extracted the marks of 10 students. 6 students scored above 60 in at least one subject. The percentage is 60%. The verified teacher-side execution completed with 1 test, 0 failures, 0 errors and BUILD SUCCESS.

23. ERR_CONNECTION_REFUSED — Important Troubleshooting

If you run mvn test before starting the server, Selenium can report:

```
org.openqa.selenium.WebDriverException:
```

This means nothing is listening on localhost:8000.

11. Keep or return to Terminal 1.
12. Run python3 -m http.server 8000.
13. Keep Terminal 1 open.
14. Open Terminal 2.
15. Run mvn test again.

24. Other Common Problems

A. Maven not found

```
mvn -version

sudo apt-get update

sudo apt-get install -y maven
```

B. Java not found

```
sudo apt-get update

sudo apt-get install -y openjdk-21-jdk
```

C. No POM found

You are in the wrong directory. Run:

```
find .. -name pom.xml
```

Move to the project directory containing pom.xml, then run mvn test.

D. Browser/driver problem

```
which chromium

whichchromium-browser
which google-chrome
which chromedriver

sudo apt-get update
```

E. SessionNotCreatedException

The working Codespaces configuration uses headless/container options. Do not remove them from the supplied program:

```
--headless=new
--no-sandbox
--disable-dev-shm-usage
```

F. CDP Warning

You may see a warning such as 'Unable to find CDP implementation matching 151'. In the verified teacher run, this warning did not prevent execution. The test still completed with 0 failures, 0 errors and BUILD SUCCESS. Do not add extra dependencies solely because of this warning unless instructed.

25. Final Student Checklist

- ☐ Opened the teacher repository.
- ☐ Created/opened the GitHub Codespace.
- ☐ Confirmed Java with java -version.
- ☐ Confirmed Maven with mvn -version.
- ☐ Confirmed pom.xml is present.
- ☐ Confirmed students.html is present.
- ☐ Started python3 -m http.server 8000.
- ☐ Kept the server terminal open.
- ☐ Opened a second terminal.
- ☐ Ran mvn test from the project directory.
- ☐ Verified Total Students: 10.
- ☐ Verified qualifying students: 6.
- ☐ Verified Percentage: 60.0%.
- ☐ Verified Tests run: 1, Failures: 0, Errors: 0.
- ☐ Verified BUILD SUCCESS.

26. Practical Record Result

Result: The student marks were successfully extracted from the dynamic web table using Selenium WebDriver. 6 out of 10 students scored above 60 in at least one subject, giving a percentage of 60%.

Name: Palak Singh Roll no. 202403123 TYCS-B STQA

Analysis result:

```
========================================
          ANALYSIS RESULT
========================================
Total Students: 10
Students scoring above 60 in at least one subject: 6
Percentage: 60.0%
========================================
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 6.492 s -- in practical.StudentScoreTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  20.160 s
[INFO] Finished at: 2026-08-20T03:17:01Z
[INFO] ------------------------------------------------------------------------
```

Output:

==========

STUDENT SCORE ANALYSIS

==========

Roll No: 101

Name: Aarav

English: 72

Mathematics: 55

Science: 68

Status: Above 60

Roll No: 102

Name: Diya

English: 45

Mathematics: 52

Science: 58

Status: Below 60

Roll No: 103

Name: Rohan

English: 81

Mathematics: 67

Science: 74

Status: Above 60

Roll No: 104

Name: Ananya

English: 55

Mathematics: 61

Science: 49

Status: Above 60

Roll No: 105

Name: Kabir

English: 39

Mathematics: 48

Science: 52

Status: Below 60

Roll No: 106

Name: Meera

English: 65

Mathematics: 58

Science: 62

Status: Above 60

Roll No: 107

Name: Rahul

English: 51

Mathematics: 44

Science: 59

Status: Below 60

Roll No: 108

Name: Isha

English: 76

Mathematics: 69

Science: 71

Status: Above 60

Roll No: 109

Name: Arjun

English: 42

Mathematics: 57

Science: 46

Status: Below 60

Roll No: 110

Name: Sara

English: 63

Mathematics: 54

Science: 60

Status: Above 60

```
========================================
          ANALYSIS RESULT
========================================
Total Students: 10
Students scoring above 60 in at least one subject: 6
Percentage: 60.0%
========================================
```